

RhythmX Intelligence

Detailed Architecture Design

A production-oriented design for authenticated wellness analytics, trusted trend retrieval, and constrained AI explanations.

Architecture principle	Decision
Facts before language	PostgreSQL/Timescale queries and Python calculate values; Ollama only explains the compact result.
Existing data model	Read directly from Django model-output tables. No new snapshot table is introduced.
User isolation	A gateway-authenticated numeric auth.User id scopes every health-data query.
Safety boundary	Responses are non-medical, disclose incomplete data, and never expose database internals.

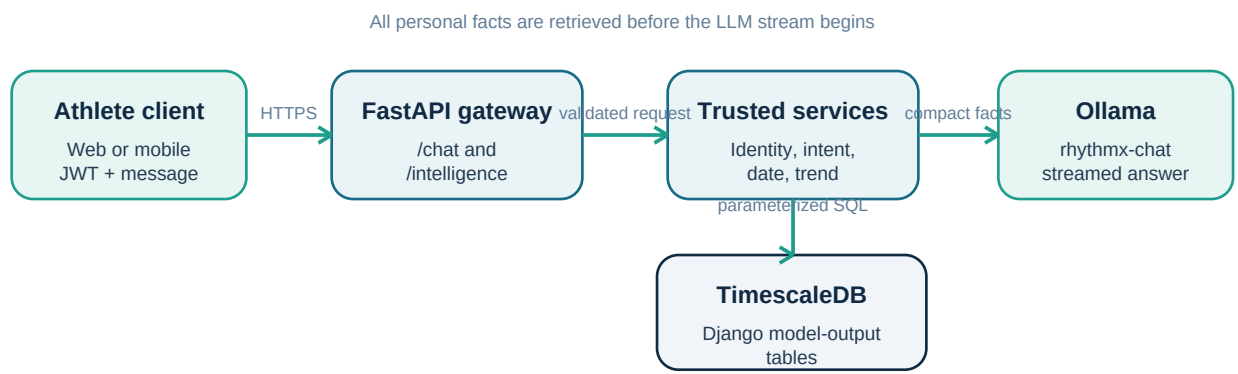
Scope

The design covers historical recovery, readiness, and injury-risk questions through the existing FastAPI service. Current-day engine endpoints remain separate and continue to call their dedicated Ollama models.

Design baseline: September 2026

1. System Context

The gateway is the single orchestration boundary. It validates the caller identity, resolves intent and dates, obtains trusted numerical history, then supplies only the resulting analytics payload to the chat model.



Component	Responsibility	Trust boundary
Client	Sends user message, timezone, and authenticated request.	Untrusted input
FastAPI gateway	Owns request validation, streaming, and service orchestration.	Application boundary
Trend service	Uses allow-listed SQL sources and computes summaries.	Trusted calculation
TimescaleDB	Stores Django-created model output records.	Private data boundary
Ollama	Turns trusted compact data into an explanation.	No database access

2. Existing Data Architecture

The service reads existing Django/Timescale model-output tables. Each query is constrained by user_id, date range, and a fixed metric-to-source mapping. Because source tables may have more than one record per date, the SQL selects the latest time value for each day.

Chat metric	Existing table	Daily value	Supporting fields
Recovery	ts_model_output_recovery_score	recovery_score	hrv_score, sleep_score, fatigue_score, interpretation_band, key_drivers
Readiness	ts_model_output_readiness_index	readiness_score	ri_base, ri_final, readiness_zone, penalty_triggers, key_drivers
Injury risk	ts_model_output_risk_score	injury_risk_prob	risk_band, risk_zone, model_version, key_drivers

Daily retrieval pattern

SELECT DISTINCT ON (date) ... WHERE user_id = \$1 AND date BETWEEN \$2 AND \$3 ORDER BY date, time DESC.
This retains one latest model output per athlete per local calendar day, while the outer query restores chronological order.

Data rule	Implementation
Identity	X-Athlete-ID is validated as a positive integer compatible with Django auth.User.
Source control	Table and column identifiers originate only from the METRICS allow-list, not client text.
Null handling	Null daily scores are excluded; availability is surfaced in data_quality.
Risk semantics	A declining injury-risk probability is interpreted as improving. A 0-1 probability uses a 0.03 stability threshold; score-scale risk uses 3 points.

3. Request and Analytics Flow

Retrieval completes before response streaming begins. This avoids partial answers based on unverified facts and keeps the LLM out of data-access decisions.



Stage	Input	Output
Authorization	X-Athlete-ID from trusted gateway	Validated Django user_id
Intent and date resolution	Natural-language message and timezone	Allow-listed metric plus calendar range
Trend query	user_id, metric source, dates	Chronological latest-daily values
Summary	Daily values	Average, range, change, direction, completeness
Prompt assembly	User question + analytics payload	Bounded facts-only LLM context
Streaming	Ollama chat response	Concise non-medical explanation

Example analytics contract

The chat model receives a JSON payload containing metric, label, period, daily_values, summary, comparison when requested, data_quality, and source_table. It does not receive a SQL connection, unfiltered records, or authority to calculate missing data.

4. API and Deployment Design

The FastAPI application exposes an authenticated chat stream, separate current-day intelligence endpoints, and a read-only database health probe.

Endpoint	Role	Key controls
POST /chat/	Historical trends and comparisons	Trusted identity header, fixed intent parser, parameterized queries, analytics context
POST /intelligence/{engine_type}	Current-day recovery/readiness/risk/overall model outputs	Allow-listed engine name; no score-history retrieval
GET /health/db	Operational readiness	SELECT 1 only; no credentials or health data returned

Recommended deployment topology

Layer	Recommended implementation
Edge	TLS terminator / API gateway validates JWT and injects X-Athlete-ID.
Application	FastAPI container with pooled asyncpg connections and environment-provided DATABASE_URL.
Database	PostgreSQL with TimescaleDB; TLS required; least-privilege read account for the chat service.
Model runtime	Ollama instances with separate chat, recovery, readiness, risk, and overall model mappings.
Observability	Request correlation, latency, token metrics, error class logging, and an audit store that avoids raw sensitive records.

Network boundary: only the FastAPI service can reach the database; the Ollama runtime receives serialized, minimal context rather than database credentials or direct connectivity.

5. Security, Reliability, and Delivery

Health-adjacent analytics requires strong separation between user-controlled text, trusted calculations, and the model response.

Control	Required behavior
Authorization	Validate JWT upstream; bind X-Athlete-ID to the authenticated subject; never accept a user id from model output or arbitrary payload fields.
Database access	Use TLS, a least-privilege role, parameterized values, and an allow-list for identifiers. Consider database RLS as defense in depth.
LLM safety	System message prohibits inventing dates or scores, medical diagnosis, raw database exposure, and missing-data calculation.
Privacy	Do not place raw health records in logs or runtime training datasets. Store audit metadata and consent-aware retention controls.
Failure handling	Return 503 for unavailable database, 422 for invalid timezone/message, and clear limited-data language when days are missing.
Testing	Cover numeric identity validation, date ranges, empty results, duplicate daily records, risk direction, and cross-user access attempts.

Delivery sequence

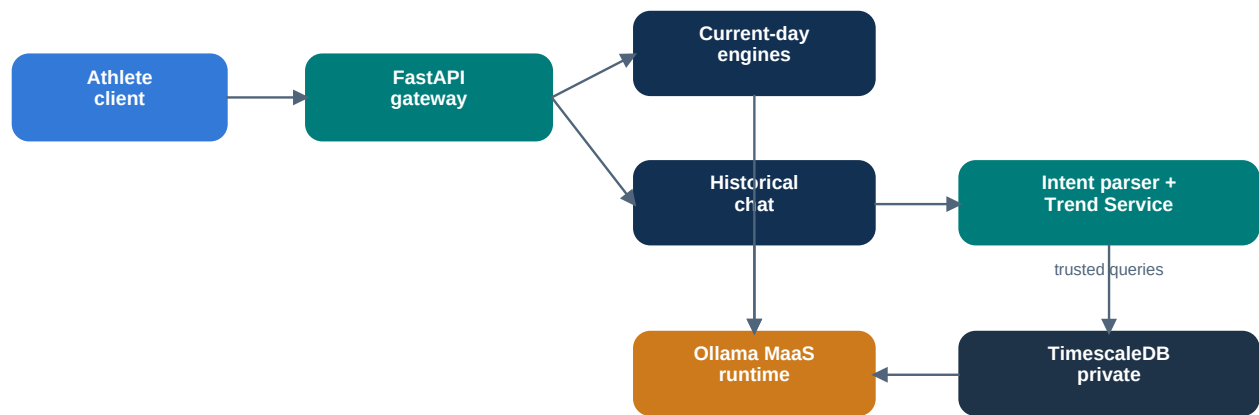
Phase	Outcome
1. Foundation	Deploy connection configuration, health probe, Django-table queries, and deterministic trend tests.
2. Experience	Add prior-period and metric comparisons, response observability, and compact conversation summaries.
3. Enrichment	Introduce permission-filtered vector context only for approved notes and only when it improves a non-numerical answer.

Final design position

RhythmX should preserve the Django application as the system of record for model outputs. The intelligence API acts as a read-only, identity-scoped analytics and explanation layer: database and Python establish facts; Ollama communicates those facts safely.

6. Model-as-a-Service architecture

RhythmX applies Prompt Microservices Architecture to athlete intelligence. Instead of one generalist prompt performing routing, data access, score interpretation, safety, and presentation, the FastAPI gateway routes each responsibility to a narrow model contract or deterministic service.



Models receive narrow, serialised context - never database credentials or raw rows

CAPABILITY	MODEL CONTRACT	RESPONSIBILITY
Chat	rhythmx-chat	Non-medical synthesis of a trusted historical payload.
Recovery	rhythmx-recovery	Plain-language recovery-driver interpretation.
Readiness	rhythmx-readiness	Capacity-to-tolerate-load interpretation.
Injury risk	rhythmx-risk	Strain/vulnerability explanation without probability exposure.
Overall state	rhythmx-overall	Cross-engine synthesis for the athlete-facing hero card.

Current state. Each tag has a separate Modelfile and task-specific instructions, but the default deployment uses one Ollama runtime and the same gpt-oss:20b base model. This provides contract isolation today; independent GPU pools and autoscaling are a later, evidence-driven deployment decision.

Safety boundary. Upstream systems calculate validated daily scores and rank drivers. MaaS prompts explain those supplied facts; they do not calculate scores, diagnose an athlete, interpret raw ECG waveforms, or access the database.

7. Trend Service and effective token utilisation

Trend Service is the historical-chat fact boundary. It resolves a supported intent and athlete-local date range, selects allowlisted fields using parameterised queries, chooses the newest value per day, calculates summaries and data completeness, then creates the analytics payload before any response token is streamed.



Goal: spend context on evidence and the final explanation, not repeated policy or model-side arithmetic

CONTROL	CURRENT BEHAVIOUR	VALUE
Deterministic routing	Known metric, date, and comparison phrases are parsed in Python.	Avoids an LLM routing call and arbitrary SQL identifiers.
Latest daily record	DISTINCT ON (date) uses the newest timestamp before chronological ordering.	Prevents duplicate data from inflating trends or context.
Server-side summary	Python calculates average, range, change, direction, and data coverage.	Prevents model-side arithmetic and invented values.
Concurrent retrieval	Primary series and supporting signals are retrieved concurrently.	Reduces database wait time on the request path.
Scoped prompt	Only recognised historical questions include ATHLETE_ANALYTICS_RESULT.	General chat avoids unnecessary retrieval and context tokens.

Token strategy. The chat prompt receives the athlete question plus a compact payload rather than raw wearable events, unfiltered database rows, SQL, or complete athlete history. Today, long periods can still grow the payload because daily values and supporting metrics are included. The next refinement is per-intent payload schemas, conditional supporting metrics, a maximum period, and measured payload/token budgets.

Measurement required. Record prompt tokens, output tokens, analytics-payload bytes, time to first token, total latency, request type, and data coverage. Do not claim a token-saving percentage until comparing an authorised compact-payload baseline with a controlled monolithic-context baseline.

8. MaaS operations, performance, and delivery

Tune each boundary at the point that owns its latency or cost. The objective is predictable specialist behaviour, efficient historical context, and safe scaling - not indiscriminate model parallelism.

LAYER	PRESENT CONTROL	NEXT TUNING ACTION
Specialist models	4,096-token contexts; task-specific prompts; bounded generation.	Align runtime output cap with 512-token Modelfile target; validate JSON and card completeness.
Chat model	8,192-token context; default 300 output tokens; low reasoning effort.	Set historical period/payload limits and monitor truncation and factual quality.
Database and trends	asyncpool plus concurrent trend retrieval.	Measure pool wait, query plans, row counts, and time spent before first model call.
GPU runtime	One private Ollama runtime, five-minute keep-alive.	Benchmark concurrency, GPU memory, and queueing before raising parallelism or splitting pools.
EC2 network	FastAPI gateway and private Ollama port 11434.	Terminate TLS at ALB/NGINX; allow least-privilege security-group paths only.

A 24 GB VRAM g6.xlarge is the documented quantised-inference baseline; use a 48 GB g6e.xlarge for demonstrated longer-context or concurrency needs. Keep model traffic private and use /openapi.json for infrastructure liveness while /health/db remains a database-readiness probe.

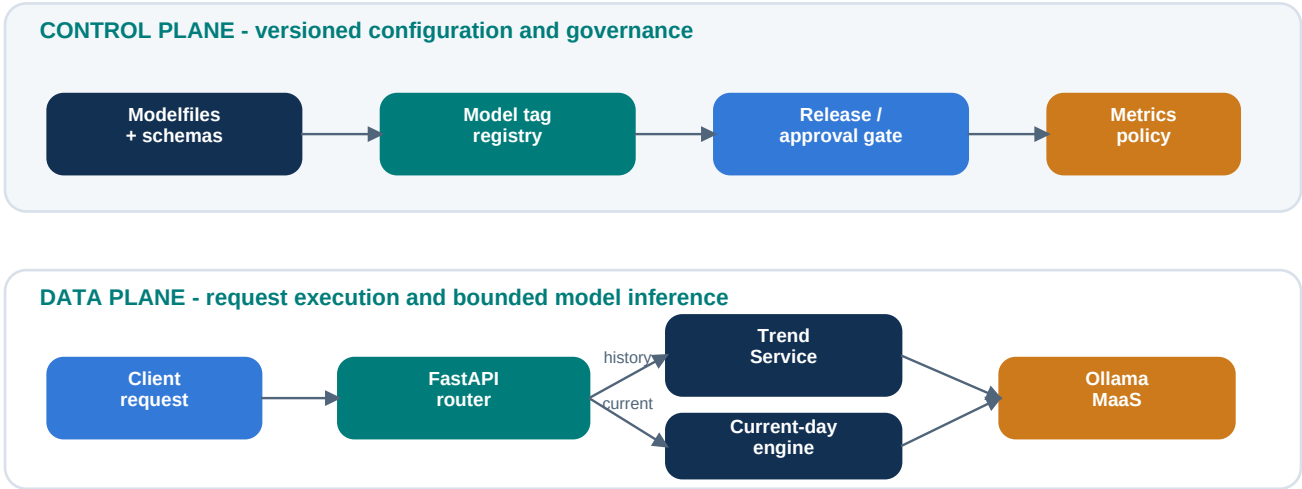
Before production health-data use, resolve athlete identity from verified authentication instead of trusting a request-body athlete_id, enforce authorisation at every query, redact sensitive logs/runtime datasets, use a least-privilege database role, and add versioned request/response schemas.

Delivery sequence: (1) secure identity and data controls, (2) validate Trend Service payload contracts, (3) add model/prompt regression tests, (4) instrument token and latency metrics, and (5) run a consented evaluation of injury-risk horizon, calibration, false alerts, and subgroup performance before external claims.

Final design position: the database and trusted application services establish facts; specialised MaaS model contracts communicate those facts safely and concisely.

9. Detailed MaaS design: control plane and data plane

MaaS is not merely a collection of prompts. It is an operating model that turns each model capability into a governed service: a stable contract, a versioned configuration, an explicit routing decision, a measurable runtime, and a safe promotion path.



Control plane changes are reviewed and promoted; data plane receives only authorised task context

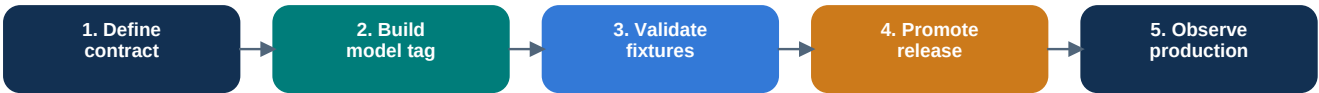
PLANE	OWNED ARTIFACTS	DECISIONS
Control plane	Modelfiles, model tags, schemas, release metadata, policy, test fixtures	Which model/version is allowed to serve which capability; parameter values; promotion/rollback.
Data plane	Request, authorised identity, validated context, Trend Service result, streamed response	Which endpoint runs; what bounded facts it receives; how failures and timeouts are handled.

In the current implementation, Modelfiles and MODEL_MAPPING provide the initial control plane, while FastAPI, Trend Service, and Ollama provide the data plane. A mature implementation should add a model registry, release approval, schema compatibility checks, and per-version telemetry.

Key rule: control-plane configuration must not be supplied by client text or inferred at runtime. The caller cannot choose an arbitrary model tag, table, column, or safety policy.

10. Endpoint contracts and lifecycle

Every MaaS endpoint should be designed as a contract with explicit inputs, outputs, boundaries, and a lifecycle. This enables a model or prompt to change without silently changing unrelated application behaviour.



Prompt + parameters + request schema + response schema + test fixtures + model tag

A prompt change is a deployable interface change, not an untracked text edit

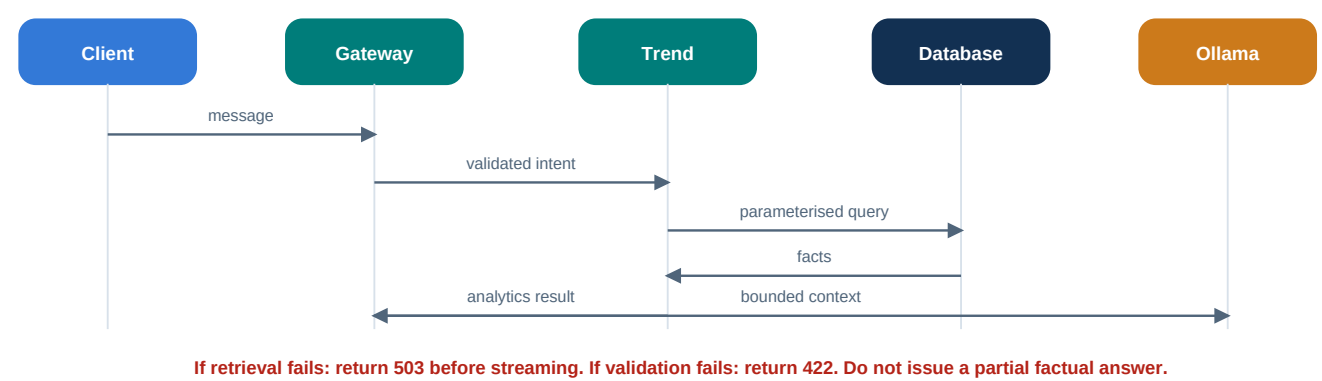
ENDPOINT	INPUT CONTRACT	OUTPUT CONTRACT	NON-NEGOTIABLE BOUNDARY
Recovery	Pre-ranked recovery drivers and band	Schema-valid card JSON	No score calculation or raw-data analysis
Readiness	Pre-ranked readiness drivers and return-to-play flag	Schema-valid card JSON	No medical interpretation; capacity framing only
Risk	Pre-ranked risk drivers and context	Schema-valid card JSON	No probability/score exposure or diagnosis
Overall	Prior engine JSON plus cross-engine drivers	Schema-valid hero-card JSON	Only documented compound-flag derivation
Chat	Question plus optional trusted analytics result	Plain-text streamed explanation	Use supplied history only; disclose data limits

Required lifecycle gates. A contract is not ready to promote until request validation, output schema checks, de-identified golden fixtures, safety/adversarial cases, token/latency budgets, and rollback compatibility have been evaluated for the proposed model tag and prompt version.

Current gap: specialist Modelfiles request JSON, but the gateway streams raw text and does not yet parse or validate that JSON. Add Pydantic input models and JSON-schema output validation before another service automates decisions from specialist output.

11. Orchestration, isolation, and failure handling

The gateway is the only orchestrator. It performs deterministic validation and trusted retrieval before the model call, so the model cannot make data-access decisions or produce a partially factual response while retrieval is still running.



FAILURE MODE	GATEWAY RESPONSE	WHY MAA S ISOLATION HELPS
Unknown model capability	404; do not call Ollama	Allowlist avoids accidental exposure of unrelated model tags.
Invalid message, athlete ID, or timezone	422 before retrieval/model call	Input failure is contained at the application boundary.
Database not configured or unavailable	503 before streaming	The model cannot invent the unavailable historical facts.
Malformed specialist response	Controlled validation error or safe retry	A broken JSON contract does not propagate silently to UI consumers.
Slow/failed model runtime	Timeout, circuit breaker, correlation ID, safe error	A model incident is observable and does not change data access policy.

Isolation principle: the database establishes facts, Trend Service packages facts, and the model explains facts. Each layer can be tested, rate-limited, scaled, or failed independently without moving database authority into the LLM.

The production target replaces body-provided athlete_id with JWT/session-derived identity and applies authorisation before every Trend Service query. This is a security requirement, not a prompt feature.

12. Model right-sizing and performance engineering

MaaS allows a workload to be tuned by responsibility. Right-sizing is a measured decision: choose the smallest model/context/output budget that satisfies factuality, safety, schema validity, user usefulness, and latency targets for a particular endpoint.

WORKLOAD	CURRENT RHYTHMX POSITION	MEASURE BEFORE CHANGING
Known intent routing	Deterministic Python parser	Ambiguous-request rate and routing errors before adding an LLM router.
Driver cards	Specialised gpt-oss:20b Modelfiles, 4,096-token contexts	JSON validity, 15-word driver adherence, latency, output-token distribution.
Historical chat	gpt-oss:20b chat, 8,192-token context, default 300 output tokens	Prompt size by period, factuality, data-quality disclosure, first-token latency.
Model pool topology	Five logical tags on one Ollama runtime	Queueing, GPU memory, fault blast radius, cost, and concurrency mix.
GPU selection	24 GB quantised baseline; 48 GB for demonstrated need	Context length, parallel requests, model loading, utilisation, and p95 latency.

Trend Service is the highest-leverage token optimisation: it eliminates raw events and model-side arithmetic. The next safe optimisation is conditional supporting metrics by intent, then explicit maximum period and payload budgets. Downsampling must preserve the requested metric and disclose aggregation/data completeness.

Instrumentation envelope: endpoint, model tag, prompt version, intent, requested days, payload bytes, prompt tokens, output tokens, time to first token, total latency, GPU memory, stream status, error class, and scoring-model version. Never log raw health content by default.

Do not increase OLLAMA_NUM_PARALLEL, keep models permanently resident, or split GPU pools solely because a setting is available. Tune against p50/p95 latency, error rate, out-of-memory events, and cost per completed request.

13. MaaS governance, security, and operating model

The service boundaries only create value when change, identity, data, and safety are governed alongside model inference. Health-adjacent intelligence requires controls beyond the prompt text.

DOMAIN	REQUIRED CONTROL	DONE-MEANS CHECK
Identity and authorisation	JWT/session-derived athlete identity; database filters/RLS; least-privilege access	Cross-athlete and unauthenticated tests cannot return another athlete's data.
Prompt release	Versioned Modelfile, schema, fixture suite, approval, rollback tag	Every deployed model tag maps to a reviewed, reproducible artifact.
Privacy	Consent, retention, deletion, redaction, encryption, audit metadata	No raw health records in normal logs or unapproved training datasets.
Observability	Correlation IDs, endpoint/model metrics, alerts, durable audit metadata	A failed request can be traced without exposing its raw payload.
Clinical claims	Consented representative evaluation, calibration/fairness review, governance approval	No one-month prediction claim without reproducible evidence.

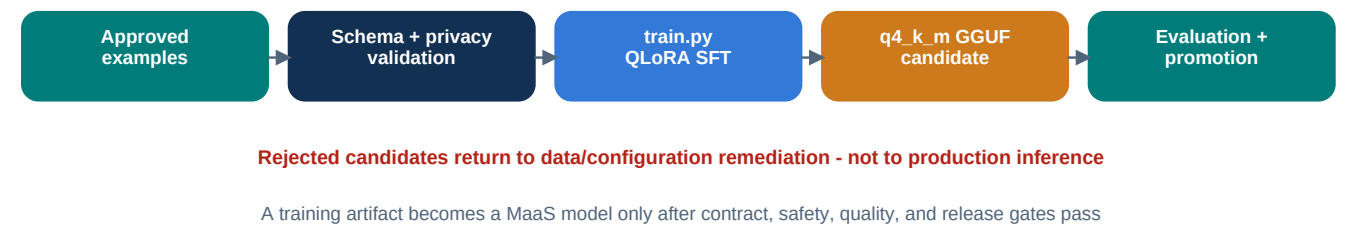
Recommended MaaS SLOs are endpoint-specific: schema-valid specialist output rate, chat factuality/data-limit disclosure, p95 time to first token, p95 total latency, database availability, GPU utilisation, and safe-failure rate. Thresholds must be set after baseline measurement rather than copied from a different workload.

- **Promote:** run contract, safety, and performance checks; approve prompt/model tag; deploy gradually; monitor versioned telemetry.
- **Rollback:** restore the last approved model tag and prompt version; preserve incident metadata; add a regression fixture before retrying.
- **Scale:** separate physical model pools only when measured queueing, cost, or isolation requirements exceed one-runtime operation.

Architecture conclusion: MaaS gives RhythmX a disciplined way to combine deterministic athlete analytics with specialised language models: facts remain in trusted services, model responsibility stays narrow, and every prompt/model change becomes an observable, testable release.

Fine-tuning and model release: governed MaaS adaptation

`train.py` adds an offline supervised fine-tuning capability. It is a candidate-model build step, not an automatic retraining or deployment mechanism. The existing Trend Service, Modelfiles, safety prompts, and release controls remain authoritative at inference time.



TRAINING COMPONENT	CURRENT CONFIGURATION IN train.py	ARCHITECTURAL INTERPRETATION
Base model	openai/gpt-oss-20b loaded in 4-bit mode	Candidate starts from the same model family used by MaaS contracts.
Sequence and adapters	2,048 max sequence; QLoRA rank 16, alpha 16, zero dropout	Memory-efficient adaptation; not proof of optimal quality or safety.
Target modules	Attention and MLP projection modules	Adapter scope is explicit and should be release-versioned.
Trainer	Batch size 2, accumulation 4, LR 2e-4, max_steps 100, seed 3407	Code defaults require reproducible experiment records and tuning evidence.
Export	rhythmx_finetuned_gguf with q4_k_m quantisation	A portable candidate artifact, not yet an Ollama tag or deployed endpoint.

Non-negotiable: Fine-tuning changes weights/adapters. Modelfiles still define the deployed task role, system safety rules, context budget, and response contract. A candidate must preserve or improve those contracts for each specialist endpoint.

Training data, evaluation, and promotion gates

The trainer expects ``dataset.jsonl`` examples with ``instruction`` and ``output``. Runtime interaction data has a different schema and can contain health-adjacent content, so it must not be used as a direct training source.

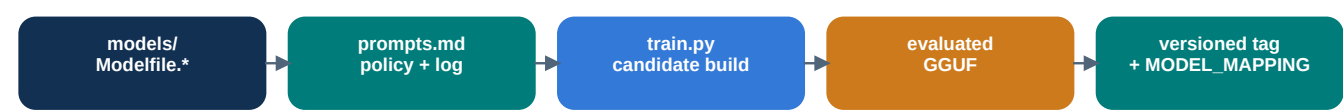
GATE	REQUIRED CONTROL	EVIDENCE BEFORE PROMOTION
Dataset lineage	Consent/authorisation, de-identification, quality filtering, immutable split and hash	Dataset card; training/validation/test manifest; owner approval.
Training reproducibility	Pinned training environment and recorded configuration	Base model, code revision, LoRA config, seed, artifact hash, quantisation.
MaaS contract	Specialist schema tests and historical chat grounding	JSON validity; disclaimer/data-limit behaviour; no SQL or model-side facts.
Safety	Adversarial and domain-specific checks	No diagnosis, raw ECG interpretation, probability exposure, or prompt-injection bypass.
Benchmark	Candidate-versus-baseline evaluation	Held-out quality, token, latency, GPU, and error-rate comparison.
Release	Versioned registration and rollback	Modelfile/tag, model card, approver, MODEL_MAPPING change, prior rollback tag.

A valid release registers the evaluated GGUF through a versioned Ollama Modelfile/tag, retains the relevant prompt contract, and updates MODEL_MAPPING only after approval. A rejected candidate remains offline. Training loss alone is never a release criterion.

Evidence status: the repository contains the QLoRA configuration and GGUF export path only. No approved training dataset, completed run, evaluation result, promoted fine-tuned model, or clinical prediction claim is recorded.

Release-policy revision: prompt and training governance

This revision consolidates prompt and fine-tuning governance into **prompts.md**. The duplicate **prompt.md** file has been removed. Canonical runtime prompts remain in **models/Modelfile.***; prompts.md records their contracts, refinements, training data policy, and release evidence.



Single source of truth for prompt governance: prompts.md

A candidate does not bypass the Modelfile contract, safety checks, or promotion gate

ARTIFACT	AUTHORITATIVE ROLE	RELEASE RULE
models/Modelfile.*	Deployed model role, safety instructions, context and generation defaults	Version with every model/prompt release.
prompts.md	Prompt ledger, fine-tuning policy, evaluation log, and promotion controls	Single documentation source for prompt and candidate-model governance.
train.py	Offline QLoRA configuration and GGUF export	Produces a candidate only; it cannot deploy or update MODEL_MAPPING.
GGUF candidate	Quantised fine-tuned artifact	Requires held-out quality, safety, schema, token/latency, and approval gates.
main.py / MODEL_MAPPING	Runtime tag selection	Update only through an approved, rollback-capable release.

Data boundary. `train.py` accepts reviewed `instruction`/`output` JSONL examples. Runtime logs have a different schema and may contain health-adjacent content; they require consent, de-identification, quality filtering, provenance, and immutable split validation before they are ever used for training.

Release boundary. Fine-tuning changes weights or adapters, while Modelfiles define the specialist MaaS contract. A model is rejected if it fails factual grounding, JSON/schema behaviour, medical-safety, prompt-injection, or data-limit disclosure checks - even if training loss improves.

Evidence status. The repository contains a QLoRA configuration and GGUF export path. It does not record an approved dataset, completed run, evaluated candidate, promoted tag, or clinical prediction result.